

Arduino Cookbook : De la Théorie à la Pratique

Chapitre 13 : Communication I2C et SPI

Présenté par : Dr B. DIOP

29 mars 2026

Référence : Michael Margolis, *Arduino Cookbook*, 2nd Edition, O'Reilly

13.0 Introduction : Les Bus de Communication

Le besoin de protocoles standardisés

Lorsqu'un projet grandit, on manque vite de broches numériques ou analogiques. Les bus I2C et SPI permettent de connecter des dizaines de composants complexes (capteurs, écrans, mémoires) en n'utilisant que 2 à 4 fils.

Dans ce chapitre, nous allons apprendre à :

- Comprendre et utiliser le bus I2C (Inter-Integrated Circuit) avec la bibliothèque Wire.
- Câbler et adresser plusieurs périphériques I2C sur le même bus.
- Configurer deux Arduino pour communiquer entre eux en I2C.
- Comprendre le bus SPI (Serial Peripheral Interface) pour la haute vitesse.
- Gérer les conflits de tensions logiques (3.3V vs 5V) sur ces bus.

13.1 Le Bus I2C : Principe et Avantages

I2C (Inter-Integrated Circuit) a été inventé par Philips. C'est un bus dit "série synchrone" fonctionnant selon le principe **Maître / Esclave**.

Caractéristiques principales :

- **Économie de broches** : N'utilise que **2 fils** de données (plus VCC et GND).
- **Adressage** : Chaque composant (Esclave) possède une adresse unique (de 0 à 127). Le Maître (l'Arduino) appelle un composant par son adresse avant de lui parler.
- **Multi-Esclaves** : On peut brancher plus de 100 composants en parallèle sur ces deux mêmes fils !
- **Vitesse** : Relativement lent (Standard à 100 kHz, Fast à 400 kHz).

13.2 Le Câblage Physique de l'I2C

Sur l'Arduino Uno, le bus I2C matériel se trouve sur les broches analogiques :

- **SDA (Serial Data)** : Broche A4. Transporte les données.
- **SCL (Serial Clock)** : Broche A5. Transporte le signal d'horloge.

Les Résistances de Tirage (Pull-up)

Le bus I2C a besoin de deux résistances de pull-up (ex : $4.7k\Omega$) reliant SDA à 5V, et SCL à 5V. **Note** : La plupart des petits modules (OLED, RTC) achetés dans le commerce intègrent déjà ces résistances sur leur circuit imprimé.

[Insérer le schéma de câblage I2C : Arduino relié à 3 modules en parallèle sur SDA/SCL]

13.3 L'outil indispensable : L'I2C Scanner

Le problème n°1 en I2C est de ne pas connaître l'adresse hexadécimale du composant que l'on vient d'acheter. L'astuce est de charger un programme "Scanner" qui va tester les adresses de 1 à 127.

```
#include <Wire.h>

void setup() {
  Wire.begin(); // Initialise le bus I2C en tant que Maitre
  Serial.begin(9600);
  Serial.println("\nI2C Scanner en cours...");
}

void loop() {
  for(byte address = 1; address < 127; address++ ) {
    Wire.beginTransmission(address);
    byte error = Wire.endTransmission(); // Verifie s'il y a une reponse

    if (error == 0) {
      Serial.print("Composant I2C trouve a l'adresse 0x");
      if (address < 16) Serial.print("0");
      Serial.println(address, HEX);
    }
  }
}
```

13.4 Faire communiquer deux Arduino en I2C

L'I2C n'est pas limité aux petits capteurs. On peut relier deux cartes Arduino ensemble !

- L'un sera le **Maître** : Il dirige la communication (demande ou envoie des données).
- L'autre sera l'**Esclave** : Il écoute, et ne parle que si le Maître l'interroge.

Règle d'or du câblage inter-cartes : Il faut **absolument** relier les broches GND (Masse) des deux cartes ensemble pour avoir une référence de tension commune, en plus des fils SDA et SCL.

Code : L'Arduino Maître (Envoie une commande)

Le Maître initialise le bus sans adresse (il est le chef) et envoie la lettre 'A' à l'Esclave situé à l'adresse 9.

```
#include <Wire.h>

void setup() {
  Wire.begin(); // Demarre en tant que Maitre (pas d'adresse)
}

void loop() {
  Wire.beginTransmission(9); // Cible l'Esclave d'adresse 9
  Wire.write('A');           // Envoie un caractere
  Wire.endTransmission();    // Ferme la communication

  delay(1000);
}
```

Code : L'Arduino Esclave (Reçoit la commande)

L'Esclave s'inscrit sur le bus avec l'adresse 9, et attache une fonction d'interruption qui se déclenchera automatiquement dès que le Maître lui parlera.

```
#include <Wire.h>

void setup() {
  Wire.begin(9);           // Demarre en tant qu'Esclave (adresse 9)
  Wire.onReceive(receptionI2C); // Definit la fonction a appeler lors d'un message
  Serial.begin(9600);
}

void loop() {
  // Rien a faire ici, la reception est geree en tache de fond !
}

// Fonction executee automatiquement lors de la reception
void receptionI2C(int howMany) {
  while (Wire.available()) { // Tant qu'il y a des donnees
    char c = Wire.read();    // Lit le caractere reçu
    Serial.print("Le maitre a dit : ");
    Serial.println(c);
  }
}
```


13.5 Le Bus SPI : La Vitesse avant tout

SPI (Serial Peripheral Interface) a été inventé par Motorola. C'est également un bus Maître/Esclave, mais conçu pour la **très haute vitesse** (plusieurs Mégahertz).

Idéal pour : Les écrans couleurs (TFT), les modules de carte SD, les modules Ethernet. Bref, tout ce qui demande de gros transferts de données.

Inconvénients :

- Utilise **4 fils minimum** (au lieu de 2 pour l'I2C).
- Ne gère pas l'adressage intégré : il faut ajouter un fil supplémentaire pour **chaque** nouvel esclave ajouté au bus.

13.6 Le Câblage Physique du SPI

Sur l'Arduino Uno, les broches matérielles SPI sont fixes :

- **SCK (Serial Clock) - Broche 13** : L'horloge générée par le Maître.
- **MISO (Master In Slave Out) - Broche 12** : Le fil où l'Esclave répond au Maître.
- **MOSI (Master Out Slave In) - Broche 11** : Le fil où le Maître parle à l'Esclave.
- **SS ou CS (Slave Select) - Broche 10 (ou autre)** : Le fil qui permet de "réveiller" l'Esclave ciblé.

[Insérer le schéma de câblage SPI : MISO sur MISO, MOSI sur MOSI]

13.7 Sélectionner un Esclave SPI (La broche CS)

Puisque le SPI n'a pas d'adresses hexadécimales intégrées comme l'I2C, comment le Maître choisit-il à qui il parle ?

La broche Chip Select (CS) ou Slave Select (SS) :

1. Par défaut, la broche CS de tous les esclaves est maintenue à l'état HIGH. Ils ignorent le bus.
2. Le Maître veut parler à l'Esclave n°1 : il passe la broche CS de l'Esclave n°1 à LOW.
3. L'Esclave n°1 "se réveille" et écoute les broches MOSI/MISO.
4. Le Maître remet le CS à HIGH une fois la transmission terminée.

Si vous avez 3 modules SPI, vous partagerez les broches 11, 12, 13, mais vous aurez besoin de 3 broches CS distinctes (ex : 10, 9 et 8).

13.8 Configurer et Utiliser le bus SPI

La bibliothèque native est `SPI.h`. Les transactions SPI modernes se configurent avec `SPISettings` pour garantir la bonne vitesse et le bon mode.

```
#include <SPI.h>

const int csPin = 10; // Broche Chip Select pour notre capteur

void setup() {
  pinMode(csPin, OUTPUT);
  digitalWrite(csPin, HIGH); // Desactive le capteur par défaut
  SPI.begin(); // Initialise les broches SPI matrielles
}

void loop() {
  // Demarre la transaction (Vitesse 1MHz, MSB en premier, Mode 0)
  SPI.beginTransaction(SPISettings(1000000, MSBFIRST, SPI_MODE0));

  digitalWrite(csPin, LOW); // 1. Reveille l'esclave
  byte reponse = SPI.transfer(0x55); // 2. Envoie l'octet 0x55 et lit la reponse
  SIMULTANEMENT !
  digitalWrite(csPin, HIGH); // 3. Rendort l'esclave

  SPI.endTransaction(); // Libere le bus
```

13.9 Comparaison directe : I2C vs SPI

Caractéristique	I2C	SPI
Nombre de fils de base	2 (SDA, SCL)	4 (MOSI, MISO, SCK, CS)
Ajout d'un périphérique	Aucun fil supplémentaire	+1 fil (CS) par périphérique
Vitesse maximale typique	400 kHz (Lent)	≥ 10 MHz (Très Rapide)
Communication	Half-Duplex (Chacun son tour)	Full-Duplex (Simultané)
Complexité matérielle	Complexe (Résistances pull-up)	Simple (Lignes directes)
Exemples d'usage	RTC, Baromètre, petit OLED	Écran TFT couleur, Carte SD

13.10 Le piège des Niveaux Logiques (Logic Level Shifting)

Attention aux composants 3.3V !

L'Arduino Uno fonctionne en **5 Volts**. Aujourd'hui, 90% des capteurs modernes I2C ou SPI (BME280, MPU6050, Cartes SD) fonctionnent strictement en **3.3 Volts**.

Envoyer un signal 5V (MOSI ou SCL) directement sur un capteur 3.3V va le détruire instantanément.

La solution : Le Logic Level Converter (Convertisseur de niveaux logiques) C'est un petit module (basé sur des transistors MOSFET) que l'on place entre l'Arduino et le capteur. Il transforme les signaux 5V en signaux 3.3V, et vice-versa, de manière bidirectionnelle.

13.11 Débogage des Bus : L'Analyseur Logique

Si votre code compile mais que le capteur ne répond pas, le problème est matériel. Comment "voir" ce qui se passe sur des fils où les impulsions durent des microsecondes ?

L'Analyseur Logique (Logic Analyzer) : C'est le stéthoscope de l'ingénieur électronicien. Contrairement à un oscilloscope qui montre les tensions (analogique), l'analyseur logique se branche en USB et décode les signaux numériques (0 et 1) pour vous montrer en clair : "L'Arduino a envoyé l'adresse 0x27, mais l'esclave n'a pas répondu (NACK)".

Un petit analyseur logique compatible "Saleae 8 channels" coûte moins de 15€ et sauve d'innombrables heures de développement en I2C et SPI.

Résumé du Chapitre 13

- **I2C (Wire.h)** : Privilégié pour économiser des broches. N'oubliez pas l'outil "I2C Scanner" pour trouver l'adresse de vos composants, et vérifiez la présence des résistances de pull-up.
- **SPI (SPI.h)** : Privilégié pour la vitesse. Les broches MISO/MOSI/SCK sont partagées, mais chaque module doit avoir sa propre broche de sélection (CS / SS).
- **Full Duplex vs Half Duplex** : Le SPI envoie et reçoit des données exactement en même temps via le registre à décalage (SPI.transfer).
- **Sécurité Matérielle** : Vérifiez systématiquement si votre module capteur tolère le 5V (5V tolerant). Si ce n'est pas le cas, utilisez un "Level Shifter".

Fin du Chapitre 13. Souhaitez-vous poursuivre avec le Chapitre 14 consacré à la Communication Sans Fil (Wireless Communication, Bluetooth, XBee) ?